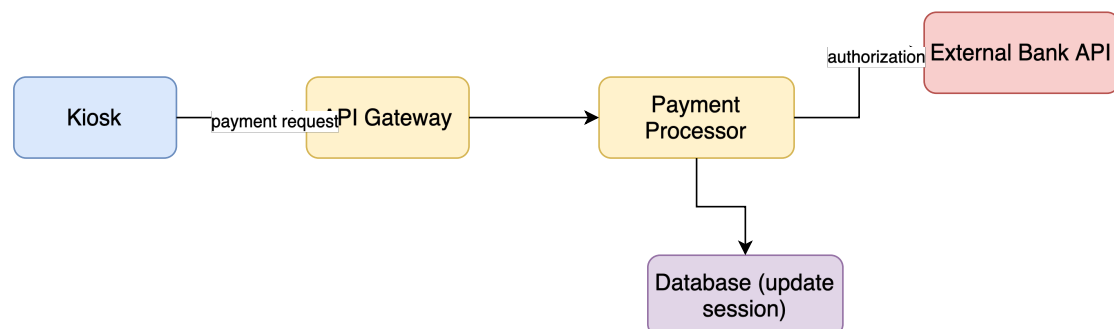


Driver and Stub Example in the Omninet AI Project

Module Under Test: Payment Processor Microservice (in the Omninet Cloud)

The Payment Processor is responsible for authorizing payments, interacting with external financial networks, and updating the session database. It receives requests from the kiosk via the API Gateway.

Call Flow for the Payment Processor



When testing the Payment Processor in isolation (component/integration testing), we need to simulate:

- **Drivers:** The components that *call* the Payment Processor (i.e., the API Gateway or a test script)
- **Stubs:** The components that the Payment Processor *calls* (i.e., the External Bank API and the Database)

Driver Example: Test Payment Driver

Purpose: Simulates the kiosk/API Gateway sending payment requests to the Payment Processor.

What it does:

- Generates various types of payment requests (cash, credit card, debit card, digital wallet)
- Sends requests with different amounts, currencies, and user data
- Simulates different network conditions (timeouts, retries)
- Verifies that the Payment Processor responds correctly (approve, reject, prompt for lower amount as per Decision Table 070-020)

Code Example (pseudo):

```
python
# Driver: TestPaymentDriver
def test_payment_scenarios():
    # Scenario 1: Valid credit card, sufficient funds
    request = {
        "payment_type": "credit_card",
        "card_number": "4111-1111-1111-1111",
        "amount": 10.00,
        "currency": "USD",
        "kiosk_id": "K001"
    }
    response = payment_processor.authorize_payment(request)
    assert response.status == "approved"
    assert response.session_minutes == 10

    # Scenario 2: Valid debit card, invalid PIN (Business Rule 4 in Decision Table)
    request = {
        "payment_type": "debit_card",
        "card_number": "5555-5555-5555-4444",
        "pin": "0000", # incorrect PIN
        "amount": 5.00,
        "currency": "USD",
        "kiosk_id": "K001"
    }
    response = payment_processor.authorize_payment(request)
```

```
assert response.status == "rejected"
assert response.reason == "invalid_pin"

# Scenario 3: Amount exceeds daily limit (Business Rule 5)
request = {
    "payment_type": "credit_card",
    "card_number": "4111-1111-1111-1111",
    "amount": 500.00, # exceeds typical daily limit
    "currency": "USD",
    "kiosk_id": "K001"
}
response = payment_processor.authorize_payment(request)
assert response.status == "prompt_for_lower_amount"
```

Why this is a Driver:

It actively calls the module under test (Payment Processor) with various inputs and checks the outputs.

Stub Example: Bank Authorization Stub

Purpose: Simulates the external bank's authorization service that the Payment Processor calls to verify funds/approve transactions.

What it does:

- Returns predefined responses based on test scenarios (approved, declined, insufficient funds, timeout)
- Allows the test to control the behavior of the external dependency without actually connecting to a real bank
- Simulates network delays and errors

Code Example (pseudo):

```
python
# Stub: BankAuthorizationStub
class BankAuthorizationStub:
    def authorize(self, card_number, amount, pin=None):
        # Predefined test scenarios
        if card_number == "4111-1111-1111-1111": # valid Visa
            if amount <= 100:
                return {"status": "approved", "auth_code": "A12345"}
            else:
                return {"status": "declined", "reason": "insufficient_funds"}

        elif card_number == "5555-5555-5555-4444": # valid Mastercard
            if pin == "1234": # correct PIN
                return {"status": "approved", "auth_code": "B67890"}
            else:
                return {"status": "declined", "reason": "invalid_pin"}

        elif card_number == "6011-1111-1111-1111": # Discover (not supported)
            return {"status": "declined", "reason": "unsupported_card"}

        # Simulate timeout for specific test case
        elif card_number == "TIMEOUT_CARD":
            import time
            time.sleep(10) # simulate delay
            return {"status": "timeout"}
```

```
# Default fallback
return {"status": "declined", "reason": "generic_error"}
```

How the Payment Processor uses this stub:

```
python
# Inside Payment Processor
def authorize_payment(request):
    # ... validate request ...

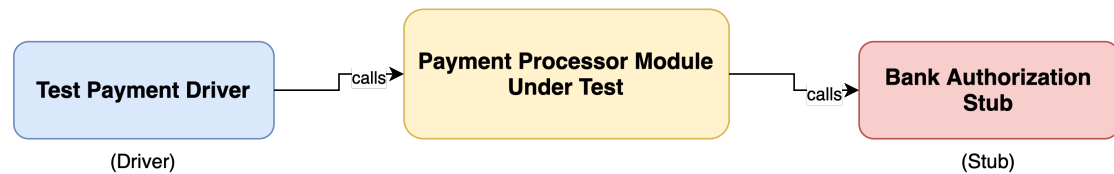
    # Call the bank stub (instead of real external API)
    bank_response = bank_stub.authorize(
        card_number=request.card_number,
        amount=request.amount,
        pin=request.get('pin')
    )

    # Process response according to Decision Table 070-020
    if bank_response.status == "approved":
        return approve_session(request.amount)
    elif bank_response.status == "declined" and bank_response.reason == "insufficient_funds":
        return prompt_for_lower_amount()
    else:
        return reject_payment(bank_response.reason)
```

Why this is a Stub:

It is called *by* the module under test (Payment Processor) and returns fake data to simulate the real external service.

Visual Summary



- **Driver** = TestPaymentDriver (simulates the kiosk/API Gateway calling the Payment Processor)
- **Stub** = BankAuthorizationStub (simulates the external bank that the Payment Processor calls)

Both are used together to test the Payment Processor in isolation, without needing a real kiosk, API Gateway, or live bank connection.